Efficient Implementation of 3G-324M Protocol Stack for Multimedia Communication*

Weijia Jia, Bo Han, Ji Shen, Haohuan Fu and Man-Ching Yuen
Department of Computer Science
City University of Hong Kong, 83 Tat Chee Avenue, Kowloon, Hong Kong, SAR China
Email: itjia@cityu.edu.hk

Abstract

In order to support real-time video, audio and data communication among heterogeneous third generation (3G) handsets, 3G phones/terminals are required to support 3G-324M, the multimedia transmission protocol stack for 3G communication. This paper discusses some efficient approaches and experiences in the implementation of 3G-324M protocol stack. Specifically, we discuss (1) Event-driven approach for the overall information exchange; (2) Single-step direct message transformation for the optimization of tree-structured message processing and (3) Serialization of nested multiplex table entries in multiplexing/ demultiplexing processing. Our implementation has been tested in a realistic heterogeneous 3G communication environment for transmission of real-time video, audio and data and its performance is satisfactory.

1. Introduction

The mobile communication market has grown with an explosive rate recent years. The number of mobile subscribers worldwide increased from 300 million in 1997 to 800 million in 2001 [1]. With wider bandwidth of the third generation wireless networks and increasing number of multimedia service categories, it can be seen that the number of subscribers is and will be having a high-speed increase, especially, when 3G is launched in the near future.

ITU-T H.324 [2] is an umbrella protocol defined by International Telecommunications Union (ITU) to enable multimedia communication over low-bit rate terminals (in the following, we will drop "ITU-T" from the prefix of standard names for simplicity). H.324 and several mobile specific annexes are usually referred to as H.324M (M stands for *mobility*). The 3rd Generation Partnership Project (3GPP) is a body that comprises wireless infrastructure, handset and service providers throughout the world. For circuit-switched 3G networks, 3GPP has adopted the H.324M with some modifications in codecs and error handling requirements to create the 3G-324M standard. In order to support the enhanced and delay sensitive video services among heterogeneous terminals, 3G phones or terminals are required to support 3G-324M protocol stack. After establishing a call, several logical channels can be set up between the call participants. Through these channels, the call control information and multimedia streams can be reliably transmitted.

A 3G-324M enabled mobile phone/terminal consists of four parts:

- Bit-stream generator—this part is an embedded software/hardware that allows the connection between two different phones/terminals.
- Signaling channel—this channel is used for the exchange of capabilities and opening of video, audio and data channels between two different phones. The signaling channel is defined by H.245 protocol.
- Data—this is the audio and video codecs and other data channels. These channels are actually what the phone users see or hear. They are also encapsulated in various standards (such as widely-known MPEG-4). Most of these solutions include software and hardware.
- Multiplexer—this software unit multiplexes and demultiplexes the signaling and data channels together to send and receive them through air interface. The multiplexer is defined by H.223 protocol.

At the low level of mobile terminals, multimedia data streams are classified as video streams, audio streams, data streams and control streams.

We have developed and implemented an efficient mobile multimedia transmission protocol stack based on 3G-324M standards in our 3G-AnyMobile project [5]. In such implementation, the performance and quality of service are critical to the success of execution of entire system. This paper discusses efficient techniques and experiences in the

* The work is supported by Research Grant Council (RGC) Hong Kong, SAR China, under grant nos.: CityU 1055/00E and CityU 1039/02E and by City University of Hong Kong Strategic grant nos. 7001587 and 7001709.

Proceedings of the 2005 11th International Conference on Parallel and Distributed Systems (ICPADS'05)
0-7695-2281-5/05 $20.00 © 2005 IEEE

implementation of 3G-324M protocol stack. Specifically, we discuss (1) *Event-driven approach* for the overall information exchange in facing hundreds of messages/ states; (2) *Single-step direct message transformation* for the optimization of tree-structured message processing; and (3) *Serialization of nested multiplex table entries* in multiplexing processing. Our implementation has been tested in a realistic heterogeneous 3G communication environment for transmission of real-time video, audio and data and its performance is satisfactory.

The paper is structured into 5 sections. Section 2 introduces the 3G-324M protocol stack. Implementations and optimizations of message transformation in H.245 and data transmission multiplexing in H.223 under 3G-324M are discussed in Section 3. Section 4 gives the performance analysis of our implementation. We summarize our major results and experience and point out the future directions in Section 5.

2. From ITU-T H.324 to 3G-324M

H.324 is a standard made by ITU-T for low bit rate multimedia communication, while H.245 and H.223 are two main parts under H.324 and have given specific descriptions about the procedures of message transformation and data transmission multiplexing. H.324 and its annex C are referred to as H.324M for mobile terminals. Thus H.324M is also an "umbrella standard" in respect with other standards which specify mandatory and optional video and audio codecs, the messages to be used for call set-up, control and tear-down (H.245 [3]) and the way that audio, video, control and other data are multiplexed and demultiplexed (H.223 [4]). H.324M terminals offering audio communication will support G.731.1 audio codec. Video communication offered in H.324M terminals will support H.263 and H.261 video codecs. H.324M terminals offering multimedia data conferencing should also support T.120 protocol. In addition, other video and audio codecs and data protocols can optionally be used via negotiation through exchange of the H.245 control messages.

The whole protocol stack of 3G-324M is shown in Figure 1. Note that the differences between 3G-324M and H.324M mainly lie in codecs (voice by AMR-Adaptive Multi Rate, and video by H.263 or MPEG-4) and error handling requirements (H.223 Annex A and Annex B as mandatory). Therefore, 3G-324M inherits H.324M basically but must use AMR for speech codec. The AMR speech coder consists of the multi-rate speech coder, a source controlled rate scheme and an error concealment mechanisms to combat the effects of transmission errors and lost packets.

3. Design and Implementation of 3G-324M Protocol Stack

In our implementation, the 3G-324M protocol stack is optimized through *event-driven approach*, *single-step direct message transformation* and *multiplex table serialization*. We first illustrate the overall operations of 3G-324M protocol and then discuss our approaches in dealing with message transformation in H.245 and data transmission multiplexing in H.223.

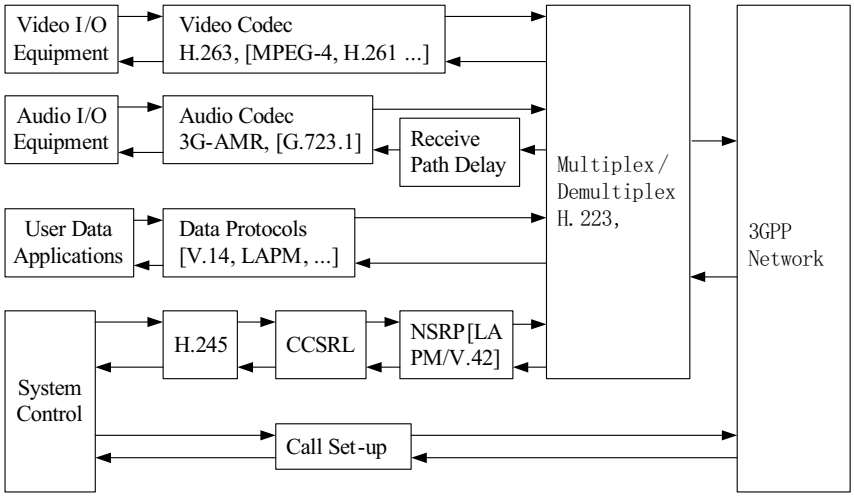


Figure 1. **3G-324M protocol stack**

Proceedings of the 2005 11th International Conference on Parallel and Distributed Systems (ICPADS'05)
0-7695-2281-5/05 $20.00 © 2005 IEEE

3.1. Control Procedure of 3G-324M

In multimedia applications of 3G mobile systems, mobile terminals must first set up the connection in general. The communication procedure between terminals can be divided into the following general steps: Phase 1. Call set-up of channels; Phase 2. Session initialization; Phase 3. Communication; Phase 4. End of session; Phase 5. Supplementary services and call clearing. The steps are further illustrated with an example as follows.

Assume terminal *A* wishes to communicate with terminal *B*. The calling terminal *A* may first set up the communication channel with *B* by entering Phase 1, in which *A* first requests the connection through some 3G wireless communication standards (air interface). After the call setup has been finished the logical channel numbered zero (LCN0) is opened for the entire period of the communication between *A* and *B*. Following this, in Phase 2, terminals' capabilities must be exchanged between *A* and *B* by transmission of H.245 *Terminal-CapabilitySet* message. This message indicates the terminal's capability to transmit and to receive the multimedia data. Thus this message should be sent first. Then, *MasterSlaveDetermination* message is transmitted, through which the terminals exchange random numbers to determine who is master and who is slave. The procedures of H.245 may then be used to open logical channels for various information streams by sending an *OpenLogicalChannel* message between the two terminals. To transmit multimedia data, some data multiplexing patterns have been defined in a multiplex table for a terminal. The multiplex table entries may be sent before or after the *OpenLogicalChannel* message. When the logical channels are opened, video, audio and data streams can be transmitted via those channels established in Phase 3 and processed through the multiplexing procedure specified in H.223. Either terminal may initiate the end of the session in Phase 4 by transmitting H.245 *EndSessionCommand* message. On subsequent receipt of *EndSessionCommand* from the remote terminal, the terminal can proceed to Phase 5. If the initiating terminal indicated an intention to disconnect the connection after the end of session, the terminal does not wait for receipt of *EndSession-Command* from the remote terminal, but can proceed directly to Phase 5.

3.2. Efficient System Control – An Event Driven Approach

To coordinate the different parts of the 3G-324M protocol stack efficiently, our implementation for 3G-324M applies event-driven programming approach which is characterized by triggering the execution of protocol in an arbitrary order, rather than in a pre-determined order such as in a sequential control procedure. This approach fits to the behaviors of network protocols as their actions may be unpredictable. Based on event-driven programming, it is easy to control the software architecture and maintain a cooperation of the protocol stack in the overall system. Figure 2 shows the event driven based system control in the protocol stack.

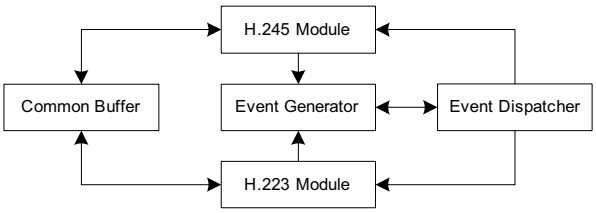


Figure 2. **Event-Driven Based System Control**

Although events are not defined explicitly in 3G-324M specification, there are generally three types of events in 3G-324M: *message*, *primitive* and *timeout* events. Message events occur when the terminal receives control messages from another terminal; primitive events happen when there is an interaction between 3G-324M and its upper layer applications; and timeout events occur when timer expires in 3G-324M protocol stack. In the *Event-Driven* control approach, two core parts are defined as *Event Generator* and *Event Dispatcher*. Event Generator generates corresponding events based on the current state and takes care of the maintenance of event queue and timer chain. Event Dispatcher calls related handling procedure based on the finite state machine definitions and it may also call Event Generator to generate other events. Our implementation uses event generator and dispatcher to coordinate H.245 and H.223 protocols to send/receive multimedia data as well as the control messages. Thus, the *Event-Driven* mechanism is executed across the entire 3G-324M protocol stack [5].

3.3. Optimized Message Processing in H.245

H.245 standard has been defined to be independent of the underlying transport mechanism, but is intended to be used with a reliable transport layer, which provides guaranteed delivery of correct data. H.245 specifies syntax and semantics of terminal control messages as well as the procedures for in-band negotiation at the start of or during communication. The messages cover receiving and transmitting

Proceedings of the 2005 11th International Conference on Parallel and Distributed Systems (ICPADS'05)
0-7695-2281-5/05 $20.00 © 2005 IEEE

capabilities as well as mode preference, logical channel signaling and control. The message syntax is defined using ASN.1 notation. For transmission, ASN.1 formatted data is transformed into bit-stream based on an ASN.1 encoding standard of Packed Encoding Rules (PER) [7].

H.245 defines a general message type *Multimedia-SystemControlMessage* (MSCM). Four types of special messages are further defined in MSCM as *request*, *response*, *command* and *indication*. In H.245, Signaling Entity (SE) is referred to as a procedure that is responsible for special functions. It is designed as a state machine and changes its current state upon reaction to an event occurrence. Taking *Master Slave Determination Signaling Entity* (MSDSE) as an example, it is a procedure for master-slave determination between two peer terminals. And it has three states: *IDLE* (no master-slave determination procedure is initiated), *OUTGOING_AWAITING_ RESPONSE* (the local MSDSE has requested a master slave determination procedure, and waits for a response from the remote MSDSE), *INCOMING_AWAITING_ RESPONSE* (the local MSDSE has received a request of master-slave determination from the remote MSDSE and sent back an acknowledgement, waiting for a response from the remote MSDSE). Figure 3 shows the workflow of MSDSE in H.245 module. Consider a new message *MasterSlaveDetermination* (MSD) is first received from the peer terminal, H.223 module demultiplexes and passes it to *Event Generator* as mentioned in Section 3.2. *Event Generator* generates an event for the message and *Event Dispatcher* recognizes it as MSD, and invokes MSDSE to handle it. Assume that the current state of MSDSE is *IDLE*, then the message event MSD triggers state dispatcher to execute the procedure *MSD_Msg_Proc*. Two parameters in the MSD message, *terminalType* and *statusDeterminationNumber* are used to determine the master slave state. If the determination succeeds, *MSD_Msg_ Proc* generates an *MSDAck* message and an Indication primitive. *MSDAck* is then sent to the far-end through H.223 module to notify the success, while the primitive is sent to the system control to indicate the success. If the determination fails, *MasterSlaveDetermination-Reject* (*MSDRej*) message is generated and sent to the far-end to notify the failure. Then *MSD_Msg_Proc* switches the current state of MSDSE to *OUTGOING_ AWAITING_RESPONSE* for waiting response. Once an event procedure is finished, H.245 module will wait for the next event.

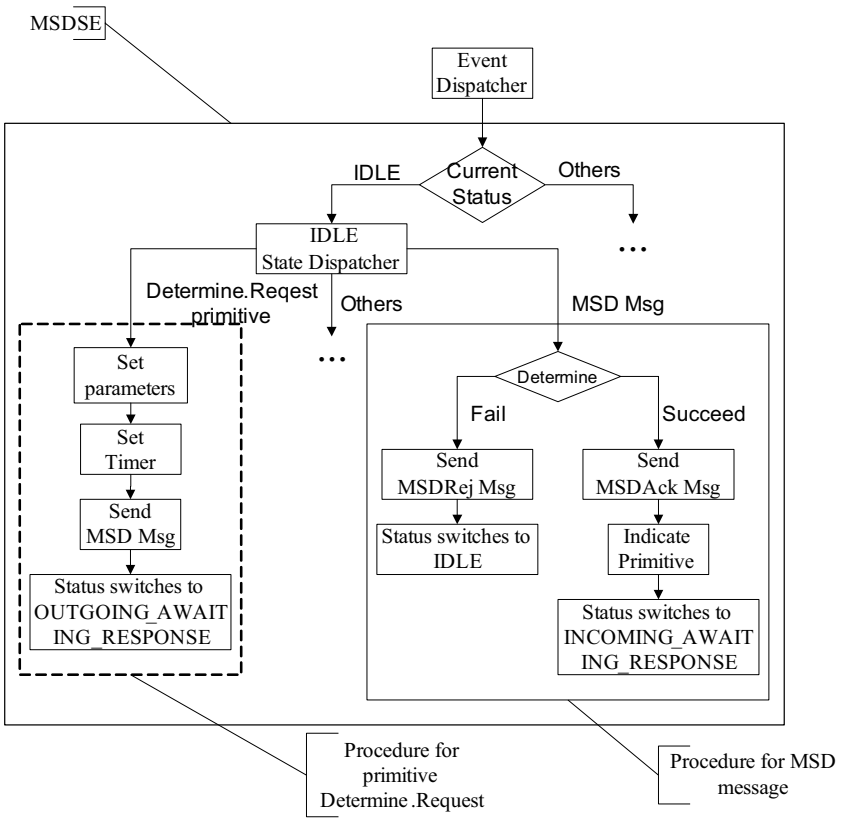


Figure 3. **The working flow of MSDSE in H.245 Module**

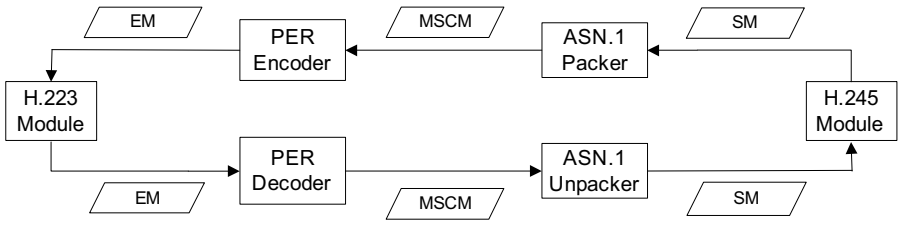
Proceedings of the 2005 11th International Conference on Parallel and Distributed Systems (ICPADS'05)
0-7695-2281-5/05 $20.00 © 2005 IEEE

Optimize message processing: Before H.245 messages are sent to H.223 module for multiplexing, all messages are expected to be packed into MSCM and then encoded by PER. Thus messages received from H.223 module should be decoded and unpacked for further processing. The implementation of packing/ unpacking and encoding/decoding involves some similar memory access and bitwise logical operations. To better utilize the limited resources of a terminal, our implementation intends to compress and integrate some of the procedures. The recent implementations of ASN.1 formatted data encoding/ decoding apply *multiple-step tree-based message transformation* [8], which organizes routines and procedures for message processing in a tree-like structure. In order to encode a H.245 message, the top-level encoding routine calls the lower level encoding routines, and the encoding routines at different levels set values to the corresponding items in a message. The processes go ahead until the messages are encoded into a bit-stream eventually. The multiple-step tree-based approach works efficiently in dealing with code redundancy and maintenance. However, the encoding/decoding operations are still complicated because the lengthy and complicated encoding rules have to be executed step by step along the tree-structure as defined in PER. In our implementtation, we propose a simple and efficient approach on implementing PER called *single-step direct message trans-formation* in which the tree-structured multi-step message encoding/decoding can be replaced by a single-step of message encoding/ decoding.

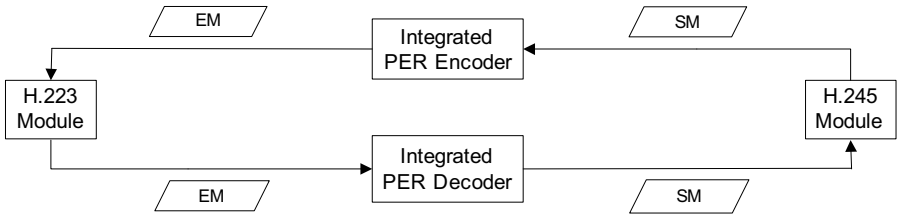
An element in a message is taken as a node and the nodes in a message are linked together into a list that represents the message. Then all messages are further linked together for quick mapping to the syntax of MSCM. In [11], PER rules are defined according to different data types, thus we have implemented the explicit encoding/decoding functions for each data type accordingly. When encoding/decoding a message, the nodes in the message list are encoded one by one. This approach is still easy to manage, and we have packed the procedures into our final implementation. As a result, based on this approach, the PER codec in our implementation is simplified without increasing the workload of encoding/decoding operations or modifying the syntax and semantics of the messages. Figure 4 shows the processing flow of our single-step implementation. In the normal operation, we need to pack a specific message into MSCM in accordance with tree-structure specification, and then the MSCM is passed to PER encoder to be encoded into a bit-stream following the working process as shown in Figure 4 (a). In our implementation, in order to simplify the process, we have combined the packing and encoding procedures into one procedure by directly transforming the message (i.e., the leaves of the tree) into a bit-stream. As a result, the message is not reduced into MSCM as specified in H.245 (Figure 4 (b)).

3.4. Multiplexing Optimization in H.223 Implementation

H.223 protocol provides low delay and low overhead by using segmentation, reassembly and combination of information from different logical channels into a single packet and it performs the multiplexing of multimedia data into bit-streams before



(a) Normal implementation approach



(b) Our implementation and optimization

Figure 4. **Characteristics of processing flow in our implementation approach (SM = Specific Message; EM = Encoded Message)**

transmission to air-interface. H.223 consists of Multiplex (MUX) Layer and Adaptation Layer (AL). AL is actually an interface for upper-layer applications and deals with different sources separately. The MUX layer performs the actual multiplexing. In this layer, data traffic from different sources can be multiplexed into one packet according to some rules which are exchanged by two terminals during the initialization of communication. These rules are described in the form of multiplex table. A multiplex table can contain at maximum 16 entries. The data structure of each entry is called a multiplex descriptor. In our implementation, the key technology for the multiplexing optimization is to apply a serialization approach, which simplifies the nested structure of a multiplex descriptor by flattening it into two linear lists. Using these two serialized lists can save much processing time during the multiplexing process and introduces less overhead for the modification of the multiplex descriptors [6].

In H.223, a multiplex descriptor is in the form of an element list. Each element in the list either represents a slot of data from a specific information source, or is extended to be a sub-element-list. An element which is not a sub-list should contain two data parts: {LCN#, RC#/RC UCF}. The first part LCN# (Logical Channel Number) specifies the information source; the second part RC#/RC UCF (RC means Repeat Count and UCF means Until Closing Flag) indicates the data slot length. For example, {LCN1, RC24} means 24 bytes from logical channel 1 that should be multiplexed into the packet and {LCN2, RC UCF} means that the bytes from logical channel 2 should be multiplexed into the packet until the end of the packet. The nested multiplex descriptor is easy to understand. However, when the descriptor structure is complicated, the processing overhead may be high as the nested form is normally processed recursively.

Serialized multiplex descriptor: To optimize the processing for the nested multiplex descriptor and avoid unnecessary recursive processing, in our implementation, a serialization approach is realized to re-structure a multiplex descriptor for efficient processing. Our approach is to identify the repeat pattern of an entry and make the right serialization processing. To identify the pattern, the key point is to understand that RC UCF only appears once in the multiplex descriptor. That is, exactly one part is repeated to the end of the packet. As a result, the whole multiplex descriptor can be divided into two parts: RC part, which is made up of elements having finite repeating count; and UCF part, which can be repeated until the end of the packet is reached. Consequently, the whole serialization process is divided into two steps. In the first step, the start point of UCF part must be identified and then the whole descriptor is divided into RC and UCF parts. In the second step, the two parts are serialized into two separate lists of *Atoms*, one list for RC part, and another for UCF part. Here *Atom* is referred to as an indivisible element which can not be extended into a sublist. The *Atom* structure contains three members: a *logical channel number* that specifies the information source; a *repeat count* which is a finite number that specifies how many bytes from that source will be filled in a packet; a *pointer* which links to the following atom. After the serialization process, the complicated nested table entry structures can be transformed into two straightforward linked lists of *Atoms*. The serialization process is shown in Figure 5.

Comparison between original and serialized multiplex descriptor: The serialization of multiplex descriptor will introduce additional cost. This cost must be taken into consideration. In general, the serialization process may be invoked before the actual data communication; it may be counted as the initialization cost. With the original nested table entry structure, recursive function call is applied to the nested multiplex descriptors. Normally, mobile terminals have

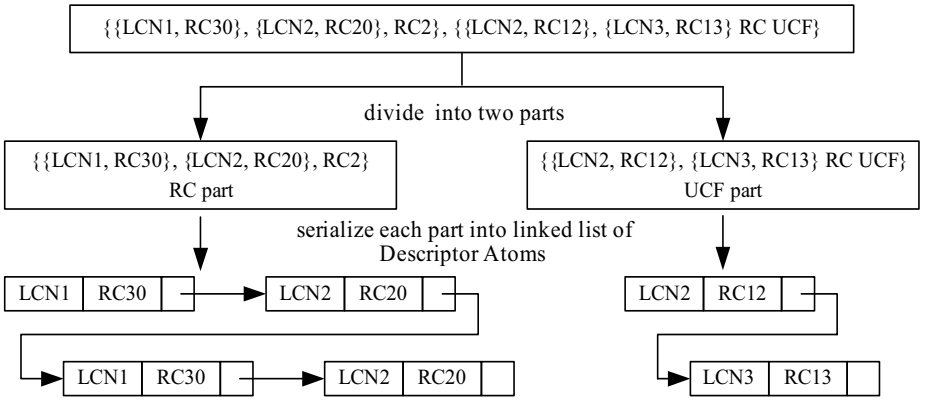


Figure 5. **The two steps of Serialization Process**

Proceedings of the 2005 11th International Conference on Parallel and Distributed Systems (ICPADS'05)
0-7695-2281-5/05 $20.00 © 2005 IEEE

limited processing-power and our purpose is to reduce the recursive invocations. With serialization of multiplex descriptors, the operation of multiplexing is straightforward with less processing overhead.

4. Performance Evaluation

In our implementation, most of the programs are implemented in ANSI C, with some key parts implemented in ARM assembly. We have tested our implementation prototype in PC (Intel P4 processor) as well as in a 3G developing board using ARM processor. We have measured some of its performance. For message conversion two measurement criteria are used to evaluate the performance: *file size* and *code size*. The file size is defined as the size of files containing source code, while the code size is defined as the size of files containing object code which is resulted from compilation. The file size and code size of the implementation of the multiple-step tree-based message transformation are 434 KB and 261 KB, respectively. In the implementation of the single-step direct message transformation, the file size is 306 KB and the code size is 221 KB. As a result, the reduction rates in file size and code size of our approach are 29.5 % and 15.3 % respectively and this is very important for a mobile terminal with limited memory.

For the data multiplexing, we have compared the performance between original and serialized multiplex descriptors. With the original descriptor, the processing cost of multiplexing one packet can be divided into two parts: C_m is the cost of filling bytes into a packet; C_n is the cost of handling the nested descriptor structure. For multiplexing of k packets, the total cost is $C(k)=k*(C_m+C_n)$. With the serialized multiplex descriptors, the processing cost of multiplexing one packet is C_m only. However, there is an additional cost to serialize the original descriptors at the initialization stage, which is defined as C_i. For the initialization stage, maximum 16 nested structures need to be handled. Thus it is reasonable to assume that $C_i \approx 16 * C_n$. For multiplexing of k packets, the total cost can be calculated as: $C(k)' = C_i + k* C_m \approx 16* C_n + k* C_m$.

We have done some experiments to evaluate the performance of our system. The result is as follows: time taken for handling a nested descriptor structure (C_n) for one million times is 0.0472s and the time taken for filling bytes into a packet (C_m) for one million times is 0.353s. From the measurement result, we can estimate that: $\frac{C_n}{C_m} \approx \frac{0.0472}{0.353} = 13.4\%$. For the multiplexing of k packets,

$$\frac{C(k)'}{C(k)} = \frac{16 * C_n + k * C_m}{k * C_n + k * C_m} = \frac{16 * 0.134 + k}{0.134k + k} \xrightarrow{k \to \infty} \frac{k}{0.134k + k} = 88.2\%$$

Thus, the overall execution cost can be optimized by 11.8% when serialization is used as compared with recursive approach.

5. Conclusions

3G-AnyMobile is a project implementing the 3G-324M protocol stack. In our implementation, event-driven approach is used for efficient control of overall protocol stack in dealing with the complex states and control transfer for the multimedia data transmission. To facilitate more efficient message transformation and data multiplexing, single-step direct message transformation and serialization approaches are applied to streamline the messages and to flatten nested structures of the multiplex table entries.

We have discussed the point-to-point multimedia transmission for 3G terminals. However, the multi-point multimedia transmission application such as video conferencing has not been addressed. The implementation of video-conferencing in 3G terminals can be a major challenge as multipoint communications require more resources. 3G-324M terminals may be used in multipoint configurations via interconnection through Multipoint Control Units. We are going to extend our implementation into a platform for video-conferencing in the 3G-AnyMobile project.

6. References

[1] ITU-R Rec. PDNR WP8F, Vision, Framework and Overall Objectives of the Future Development of IMT-2000 and Systems beyond IMT-2000, 2002.

[2] ITU-T Rec. H.324, Terminal for low bit rate multimedia communication, March 2002.

[3] ITU-T Rec. H.245, Control protocol for multimedia communication, July 2003.

[4] ITU-T Rec. H.223, Multiplexing protocol for low bit rate mobile multimedia communication, July 2001.

[5] B. Han, H. Fu, J. Shen, P. O. Au and W. Jia, "Design and Implementation of 3G-324M - An Event-Driven Approach", *Proc. of IEEE VTC'04 Fall*, Sep. 2004.

[6] W. Jia, H. Fu, B. Han and P. Au, "Efficient Data Transmission Multiplexing in 3G Mobile Systems", *Proc. of Globe Mobile Congress 2004*, Oct. 2004.

[7] ITU-T Rec. X.691, Information technology – ASN.1 encoding rules – Specification of Packed Encoding Rules (PER), 1997.

[8] Q. Ly, B. Huang, F. Wang, "The mechanism of ASN.1 encoding & decoding implementation in network protocols", *Proc. of International Conference on Information Technology: Coding and Computing (2003)*, pp. 622-626, 2003.